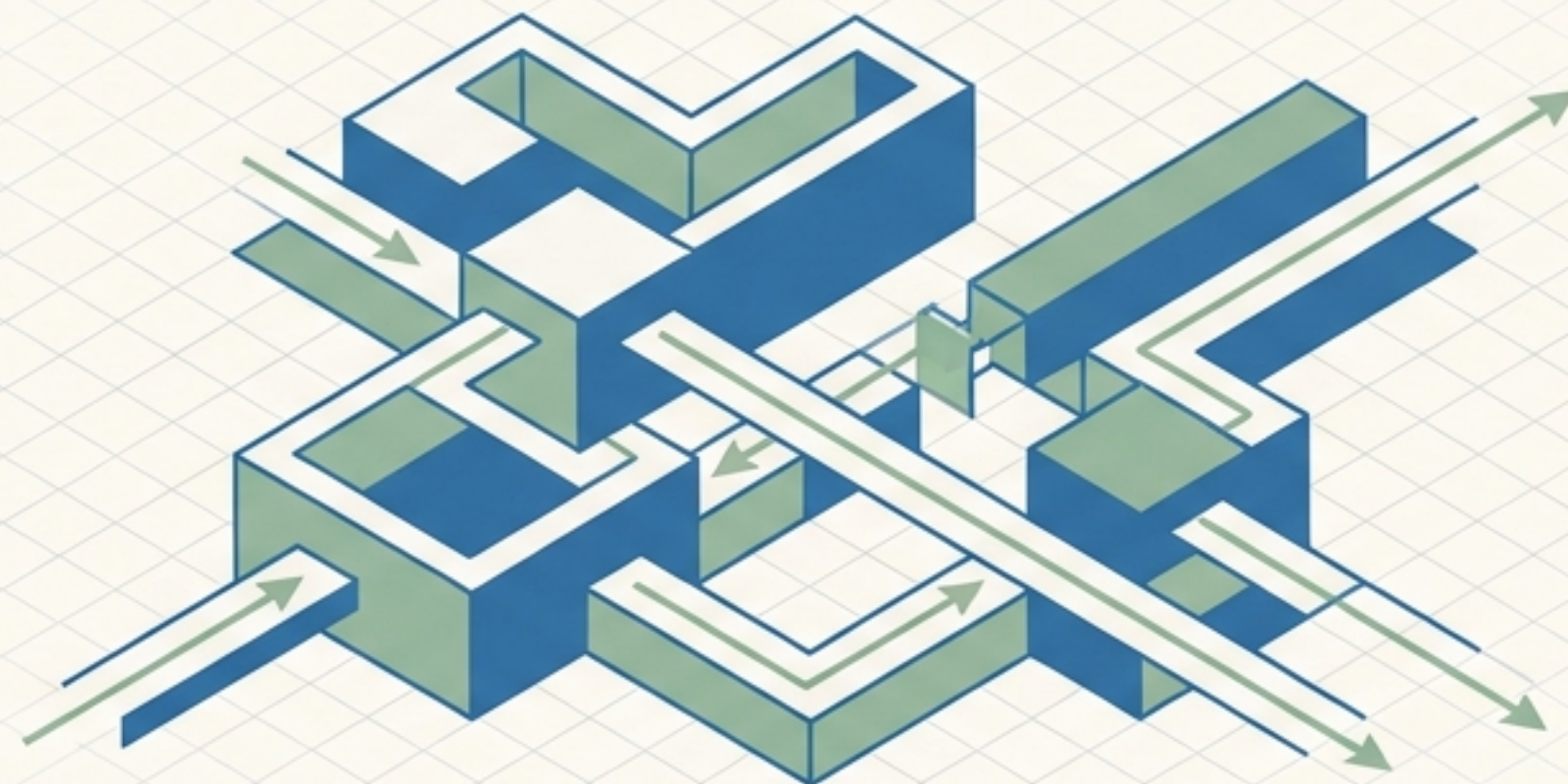


System Status: Optimized ✓ | Throughput: 10x



สถาปัตยกรรมระดับสเกล: โครงสร้างการขยายระบบ SLM ให้เร็วขึ้น 10 เท่า

เจาะลึกเบื้องหลังระบบ Semantic Job Search ของ LinkedIn ผ่านการเพิ่มประสิทธิภาพทั้ง โมเดล, ข้อมูล และ โครงสร้างพื้นฐาน

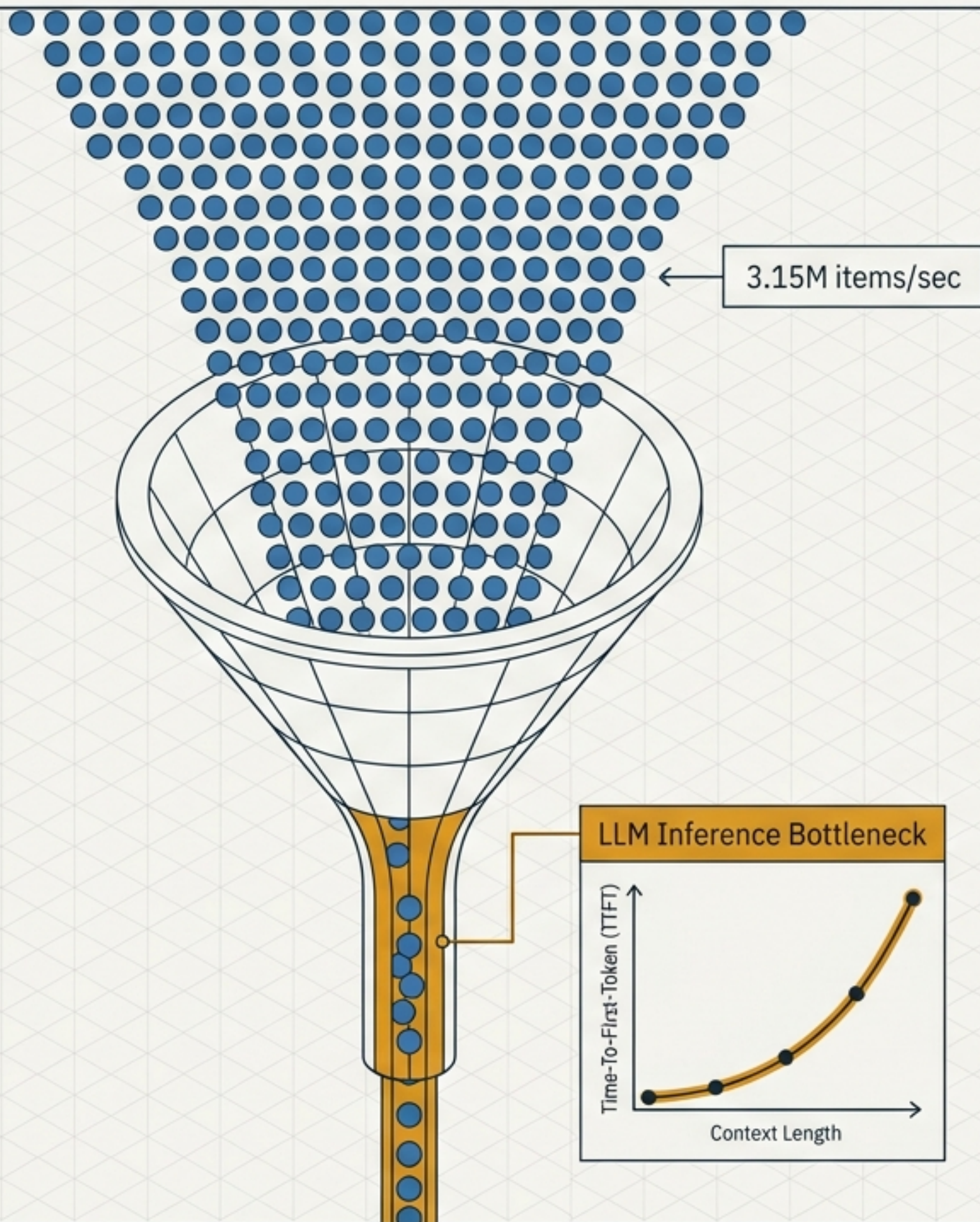
กรณีศึกษาจากวิศวกร LinkedIn

คอขวดของระบบ: ความแม่นยำที่แลกมาด้วยความล่าช้า

เป้าหมาย: ประเมินความสอดคล้องของงาน
3.15M items/sec เพื่อรองรับ Traffic มหาศาล

ปัญหา: LLM มีความสามารถในการจัดอันดับ
(Relevance Ranking) สูงมาก แต่ช้าและกิน
ทรัพยากรเกินไป

ความท้าทาย: ความซับซ้อนของ Self-Attention
ทำให้ค่า Latency (TTFT) เพิ่มขึ้นแบบ Quadratic
ตามความยาวของ Context

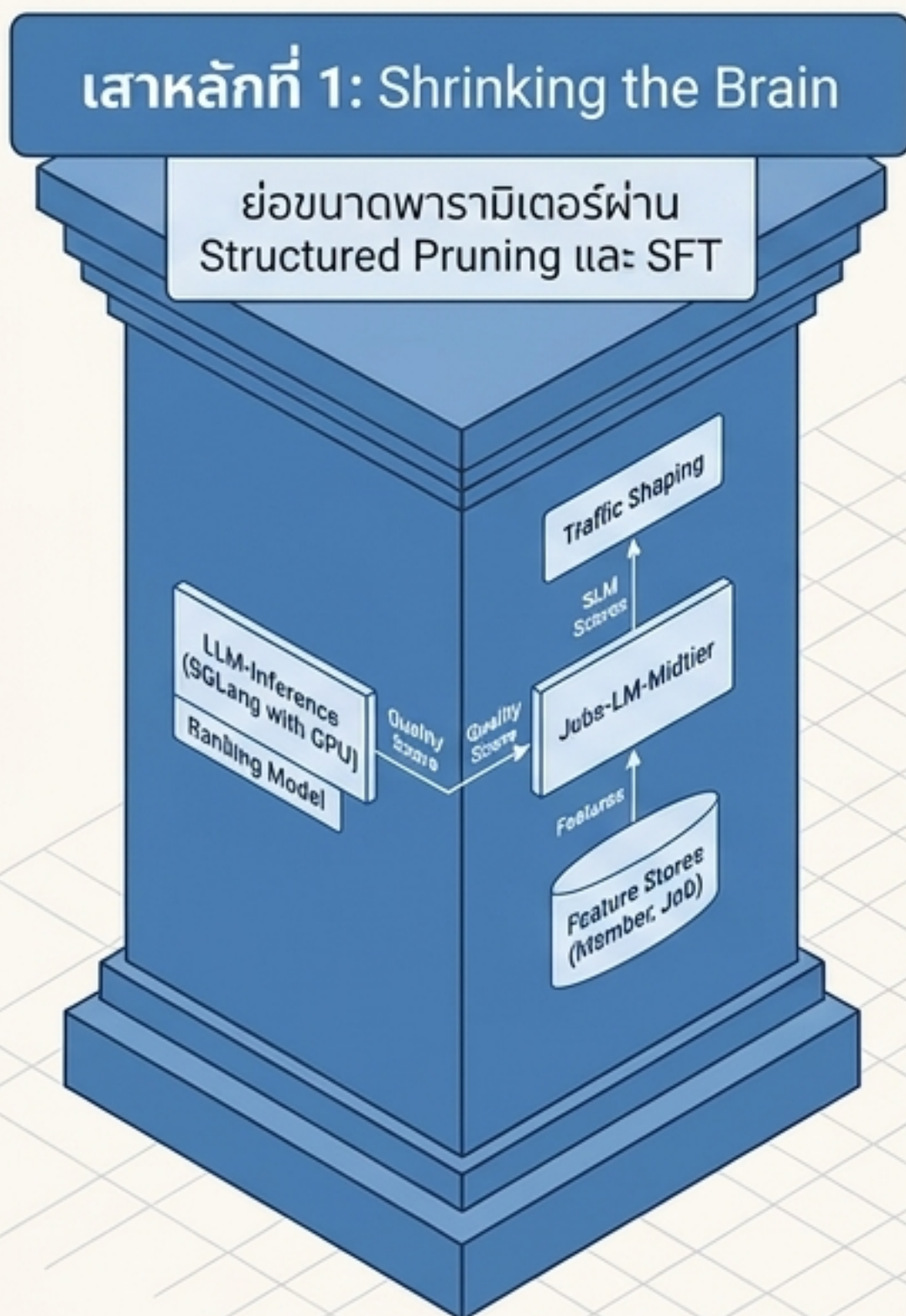


สถาปัตยกรรม 3 เสาหลักเพื่อฝ่าขีดจำกัด

System Status: Architectural View | Throughput: N/A

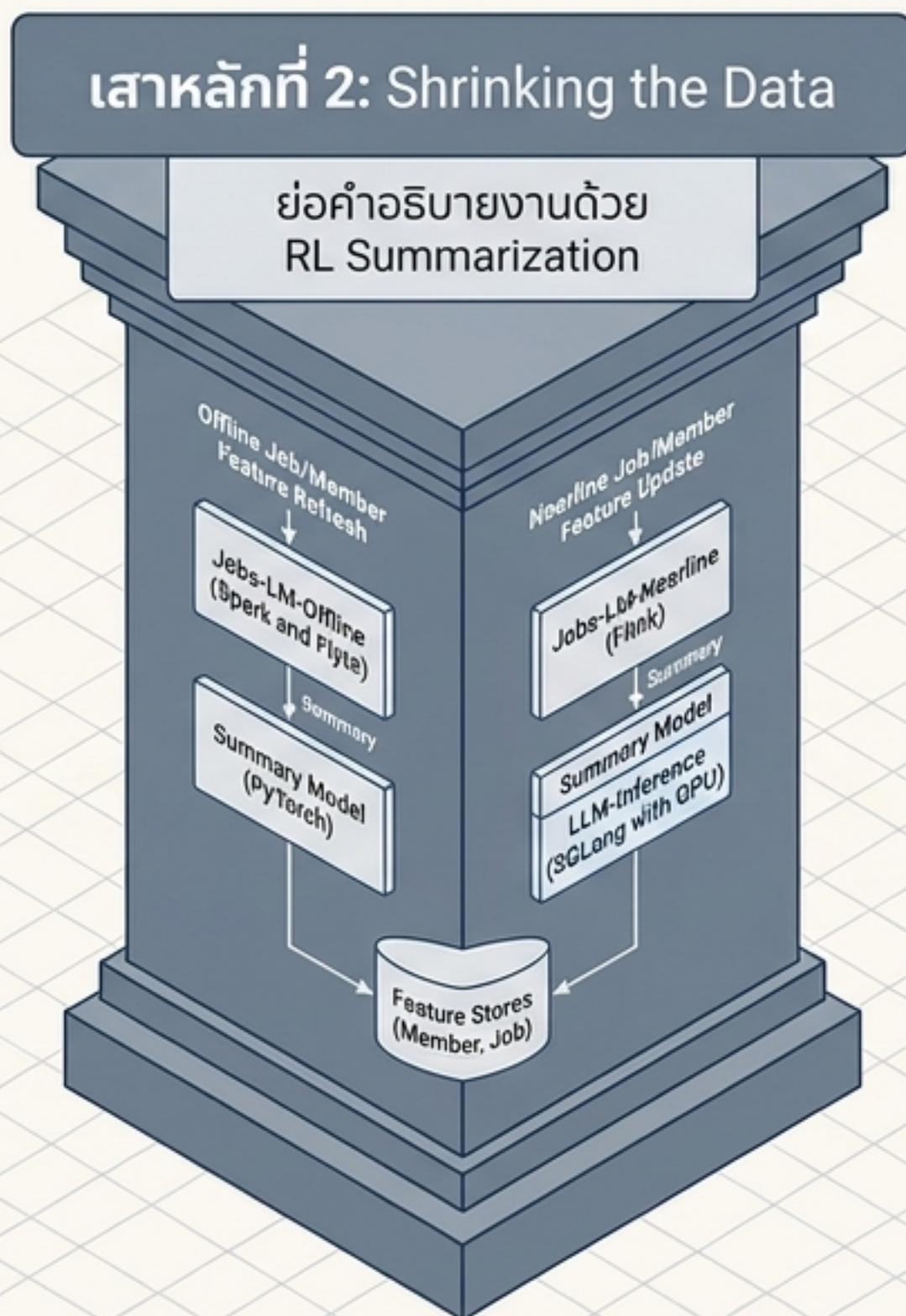
เสาหลักที่ 1: Shrinking the Brain

ย่อขนาดพารามิเตอร์ผ่าน
Structured Pruning และ SFT



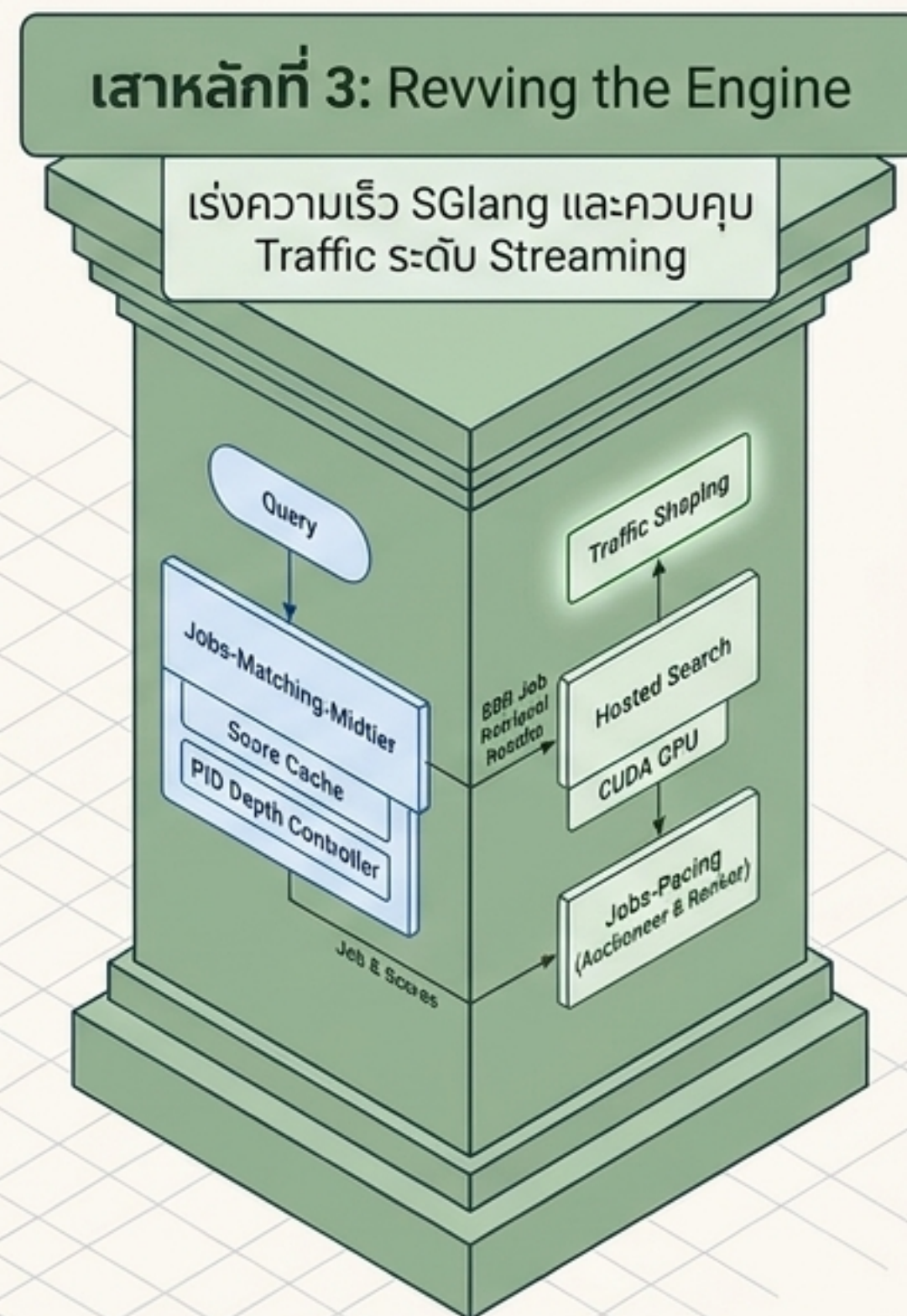
เสาหลักที่ 2: Shrinking the Data

ย่อคำอธิบายงานด้วย
RL Summarization



เสาหลักที่ 3: Revving the Engine

เร่งความเร็ว SGLang และควบคุม
Traffic ระดับ Streaming



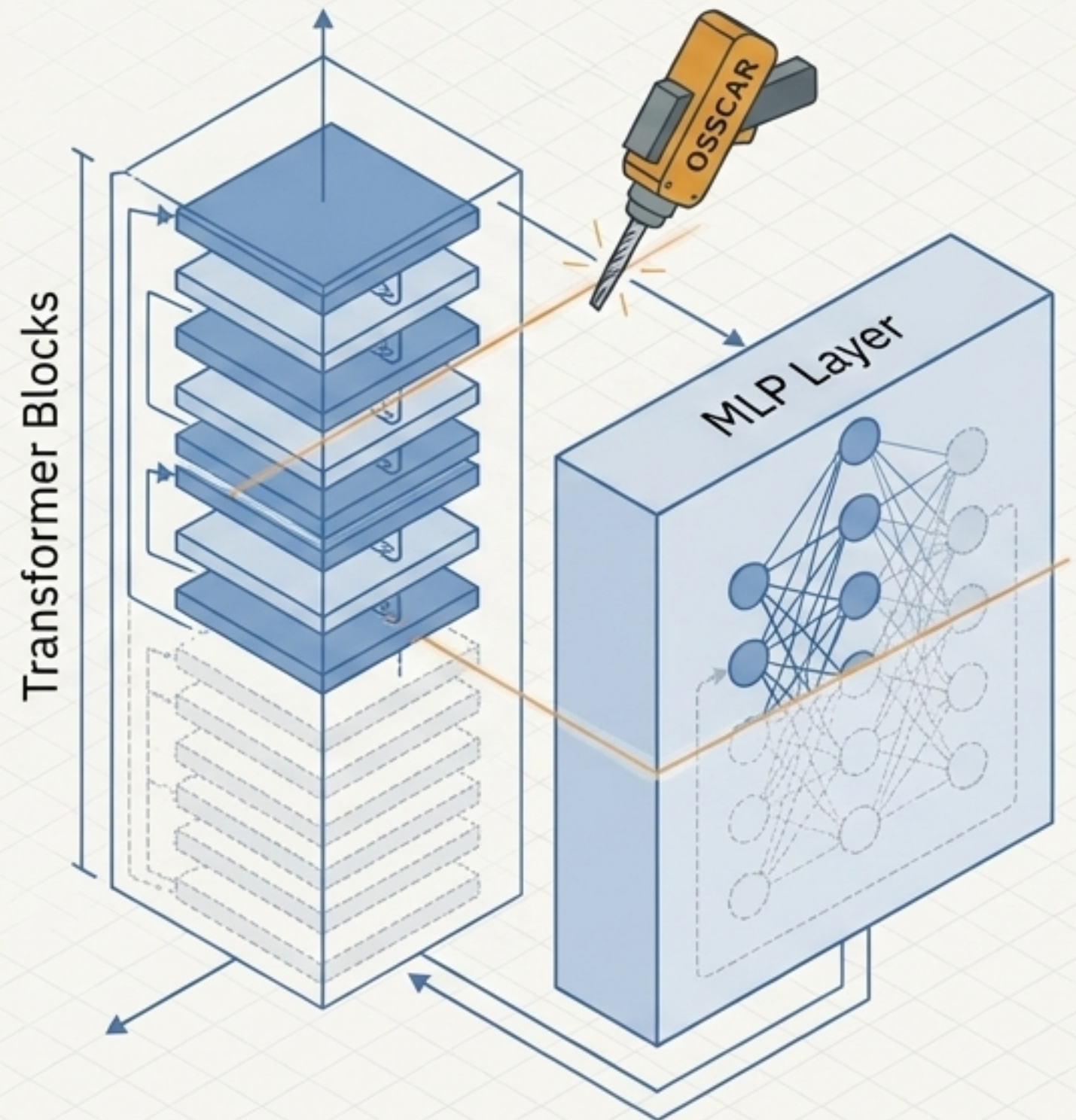
เสาหลักที่ 1: การตัดแต่งโครงสร้าง (Structured Pruning)

เทคนิค Structured Pruning ช่วยให้รันโมเดลบน GPU ทัวไปได้โดยไม่ต้องใช้ Kernel พิเศษ

ตัด Hidden Neurons ในชั้น MLP ออก 50%

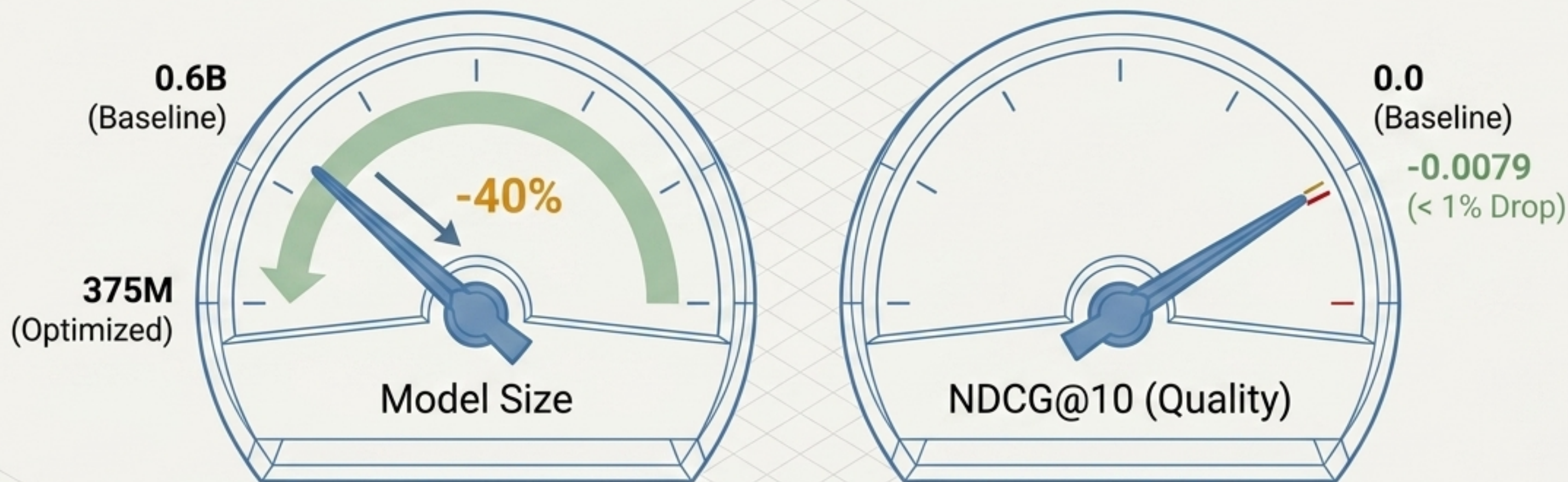
ตัด Transformer Blocks ออก 8 Layers สุดท้าย เนื่องจากพบว่ากระทบคุณภาพน้อยที่สุด

ฟื้นฟูความแม่นยำด้วย SFT (Supervised Fine-Tuning) หลังการตัดแต่ง



การแลกเปลี่ยนที่คุ้มค่า: เล็กลง 40% แต่คุณภาพคงเดิม

System Status: Pillar 1
| Throughput: 1.27x

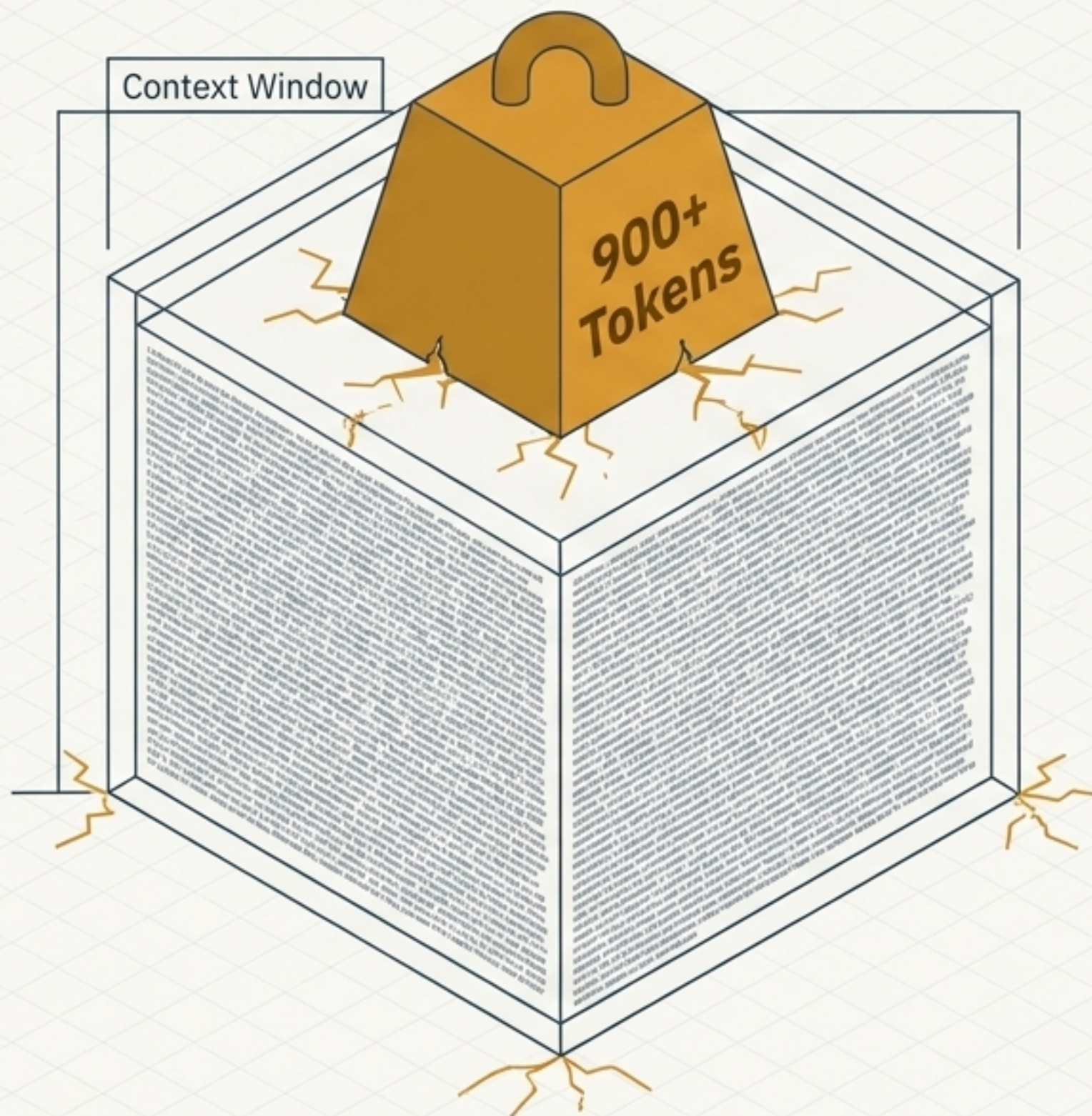


1	Baseline	0.6B Parameters NDCG Drop: 0.0
2	Optimized	375M Parameters NDCG Drop: -0.0079 (< 1%)

โมเดลเล็กลงช่วยเพิ่ม Throughput กันที่ +27% โดยคุณภาพแทบไม่ลดลง

เสาหลักที่ 2: ปัญหาข้อมูลล้น Context

System Status: Context Bloat | Throughput: 1.27x



1

คำอธิบายงานโดยเฉลี่ยมีความยาวถึง 900+ Tokens (สูงสุด >2300)

2

คิดเป็น 94% ของ Prompt ทั้งหมด ที่ส่งให้ SLM



3

ผลลัพธ์: กินหน่วยความจำมหาศาล และทำให้ Time-To-First-Token สูงจนรับไม่ได้



4

ถ้าตัดทิ้งทั้งหมด? คุณภาพ (NDCG) จะลดลงถึง -9% ทันที

ทำไม Prompt Engineering ถึงล้มเหลวในระดับสากล?

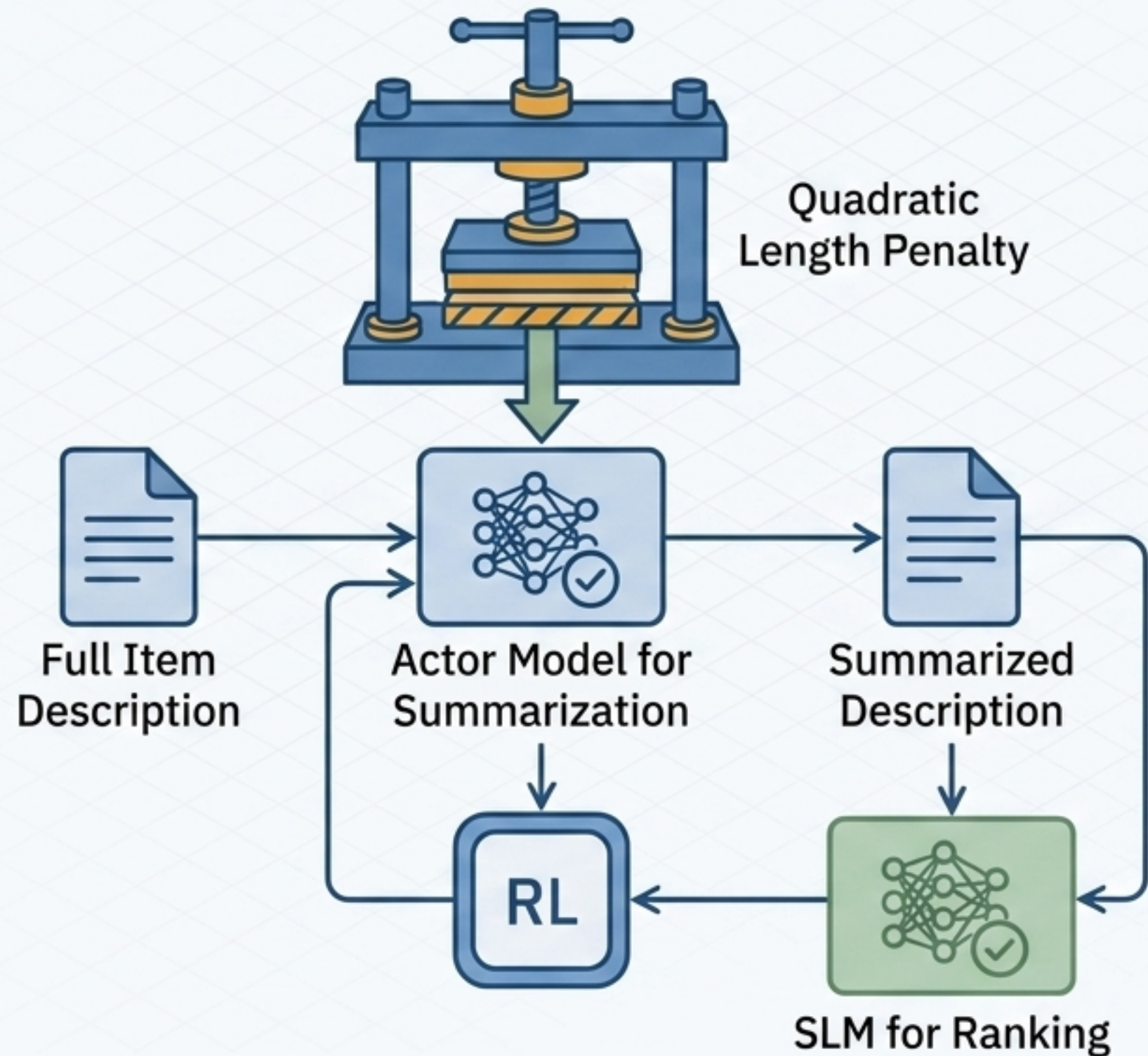
System Status: Pillar 2 | Throughput: 1.27x

	วิธี (Approach)	อัตราการบีบอัด (Compression)	คุณภาพที่ลดลง (NDCG Drop)
1	นำ Stop words ออก	-16%	-0.5% (บีบอัดได้น้อยเกินไป)
2	สั่ง Summarize	-52%	-2.5% (ตกเกณฑ์คุณภาพ)
3	สั่งจัดฟอร์แมต YAML	-74%	-2.7% (ตกเกณฑ์คุณภาพ)



Insight: Prompt Engineering ทั่วไปไม่สามารถบีบอัดเกิน **50%** โดยที่รักษาคุณภาพให้อยู่ในเกณฑ์ที่ยอมรับได้ (ดรอปร้อยเกิน **2%**)

กลไกบีบอัดด้วย Reinforcement Learning (GSPO)



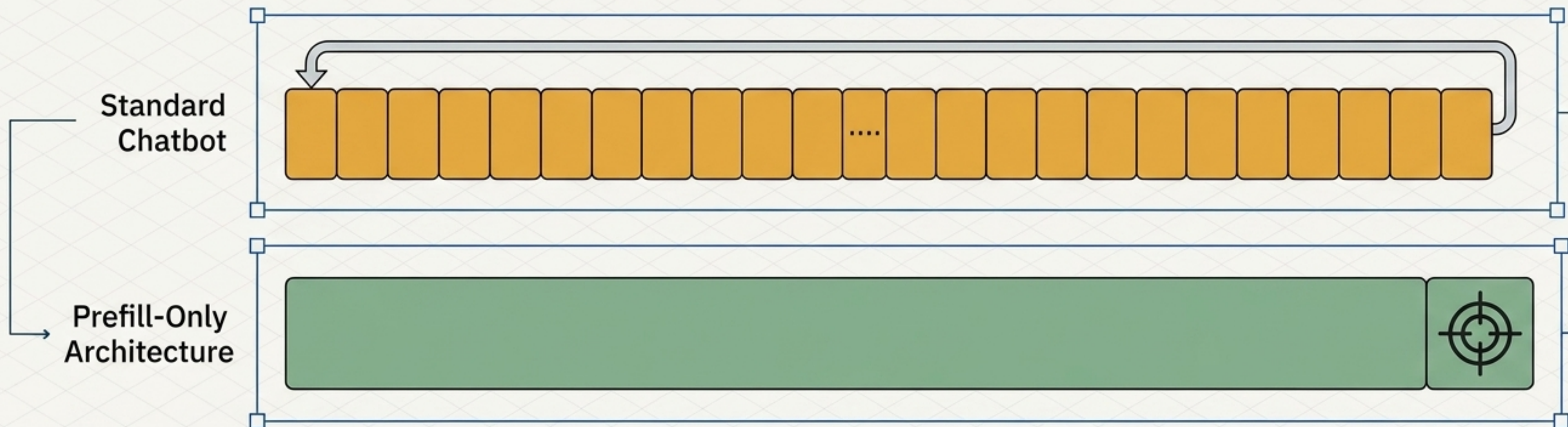
ใช้โมเดล 1.7B เป็น Actor Model เพื่อสรุปเนื้อหาล่วงหน้าแบบ Offline

ใช้กลไกการให้รางวัล (Reward) ที่ผสาน "ความแม่นยำของ SLM" เข้ากับ "บทลงโทษความยาวแบบ Quadratic"

ผลลัพธ์: บีบอัดข้อความได้ถึง 93% (เหลือความยาวเพียง 7%) โดย NDCG ลดลงเพียง -2%

เสาหลักที่ 3: ปลดล็อกเอนจินด้วย Prefill-Only

System Status: Pillar 3 | Throughput: 4.5x



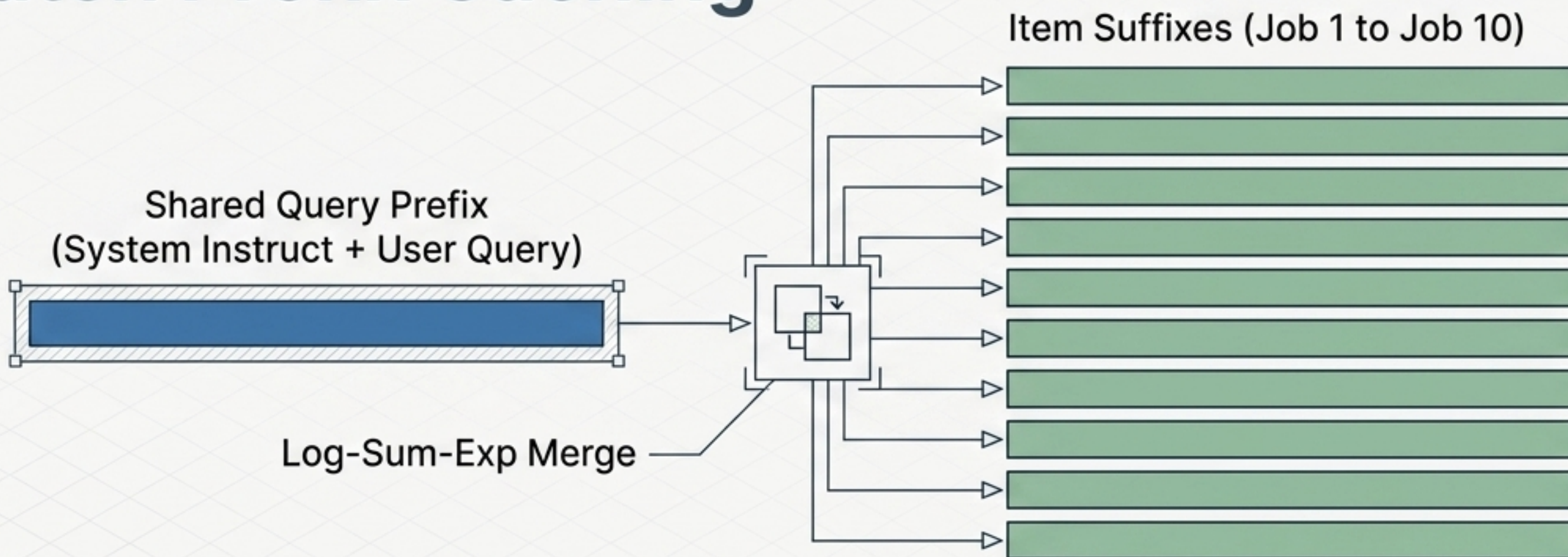
ระบบ Ranking ไม่ใช่ Chatbot!
เราต้องการแค่ความน่าจะเป็นของ
Token สุดท้าย (Yes/No)

Skip Decoding: ตัดกระบวนการ
Sampling และ Decode
ออกทั้งหมด ลด Latency จาก
→ 33ms เหลือ 20ms

Batch Tokenization: ประมวลผล
Tokenize พร้อมกันทั้ง Batch
↗ (เร็วกว่าปกติถึง 5.2 เท่า
สำหรับ Batch size 8)

ลดการคำนวณซ้ำซ้อนด้วย In-Batch Prefix Caching

System Status: Pillar 3 | Throughput: 6x



ใน 1 Request (1 Query ต่อหลายงาน) ส่วน Prefix ของ Prompt จะเหมือนกันทั้งหมด

แทนที่จะคำนวณ Attention ใหม่ทั้งหมด ระบบจะดึง KV Cache ของ Prefix จากงานแรกมาใช้ร่วมกัน

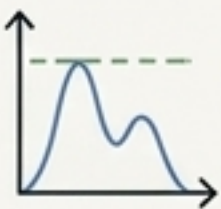
Impact: ลดการคำนวณ Token เพิ่ม Throughput ได้ถึง +33% สำหรับ Batch ขนาดใหญ่

ควบคุม Traffic อัจฉริยะ (Dynamic Ranking Depth)

System Status: Pillar 3 | Throughput: 8x



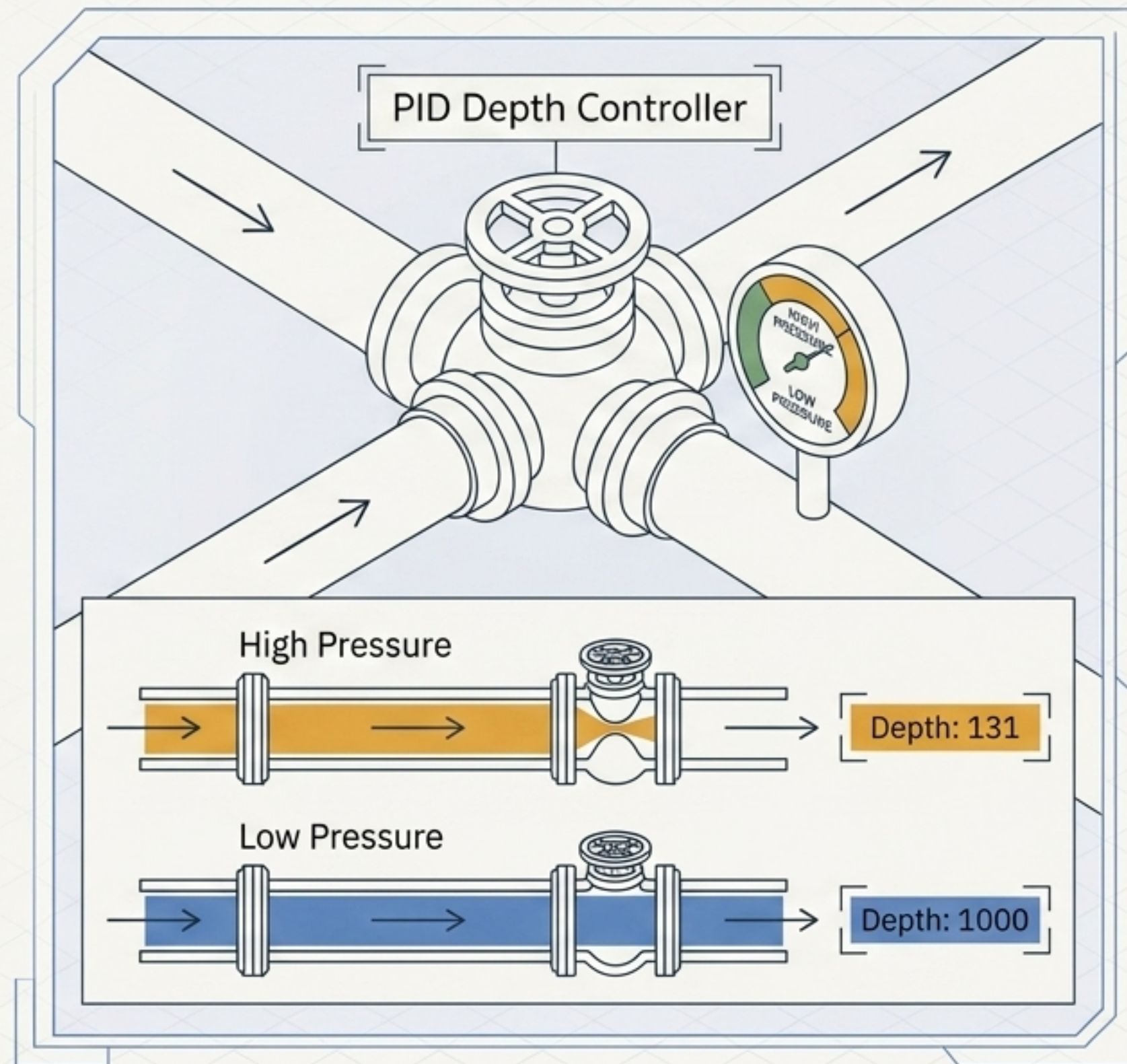
PID Controller: ปรับจำนวนผู้สมัครที่เข้าสู่ระบบ Ranking แบบอัตโนมัติตามโหลดของ GPU



Traffic Shaping: เคลียร์ Request ที่พุ่งสูง (Bursty) ไปยังช่วงที่ว่าง ลดปัญหา GPU ว่างงาน เพิ่ม Capacity ได้ 25%

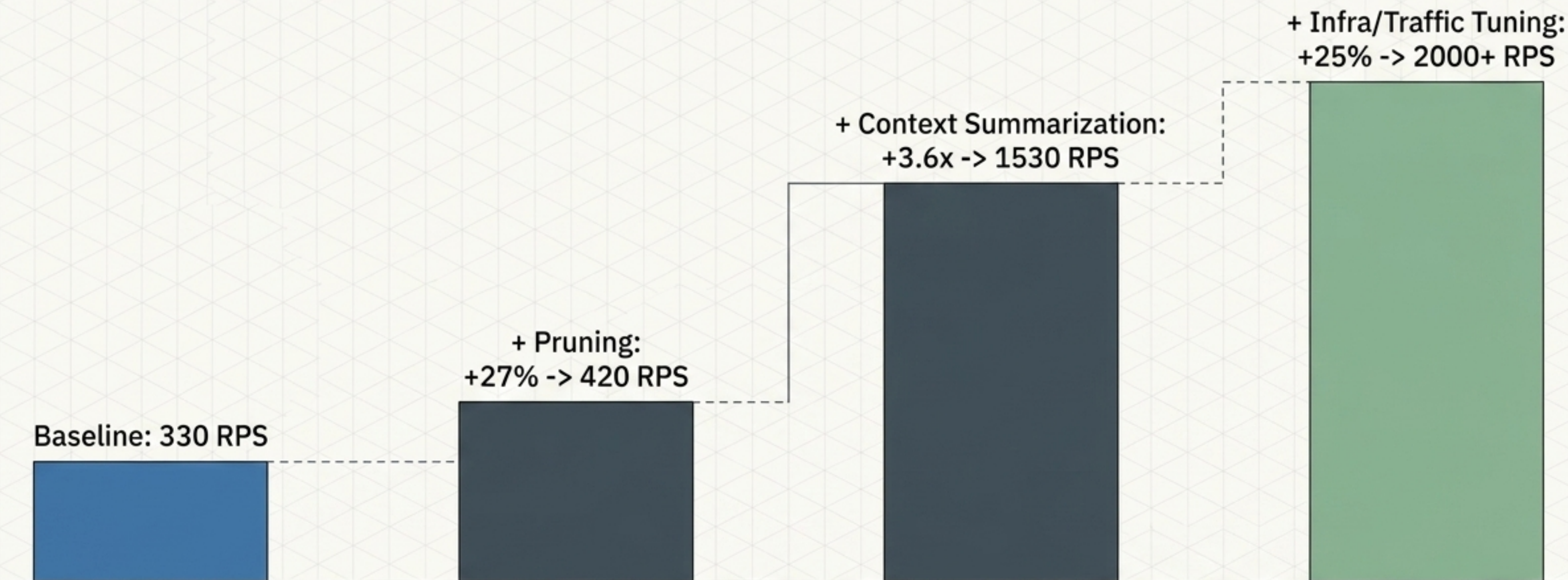


Score Caching: แคชผลลัพธ์ที่คำนวณแล้ว (Hit rate 50%) ลด Latency ลง ~10%



ผลลัพธ์แบบทวีคูณ (The 10x Multiplier Effect)

System Status: Optimized | Throughput: 10x



การแยกส่วนพัฒนาคือศัตรูของ Scale – การตัดแต่งโมเดล, การบีบอัดข้อมูล, และการปรับโครงสร้างพื้นฐาน ไม่ได้บวกเพิ่มกันแบบเส้นตรง แต่สร้างผลลัพธ์ทวีคูณ (Multiplicative) ผลักดันศักยภาพระบบจาก 330 สู่ 2000 RPS ต่อ 1 GPU

ผลลัพธ์จากการใช้งานจริง (Online A/B Test Impact)

System Status: Live | Impact: Verified

ความแม่นยำ
(NDCG@10)

+8.45%

(เทียบกับระบบ EBR เดิม)

ความผูกพันของผู้ใช้งาน
(Weekly Active Users)

+1.49%

(คันทางานสำเร็จมากขึ้น)

อัตราการปฏิเสธงาน
(Job Dismisses)

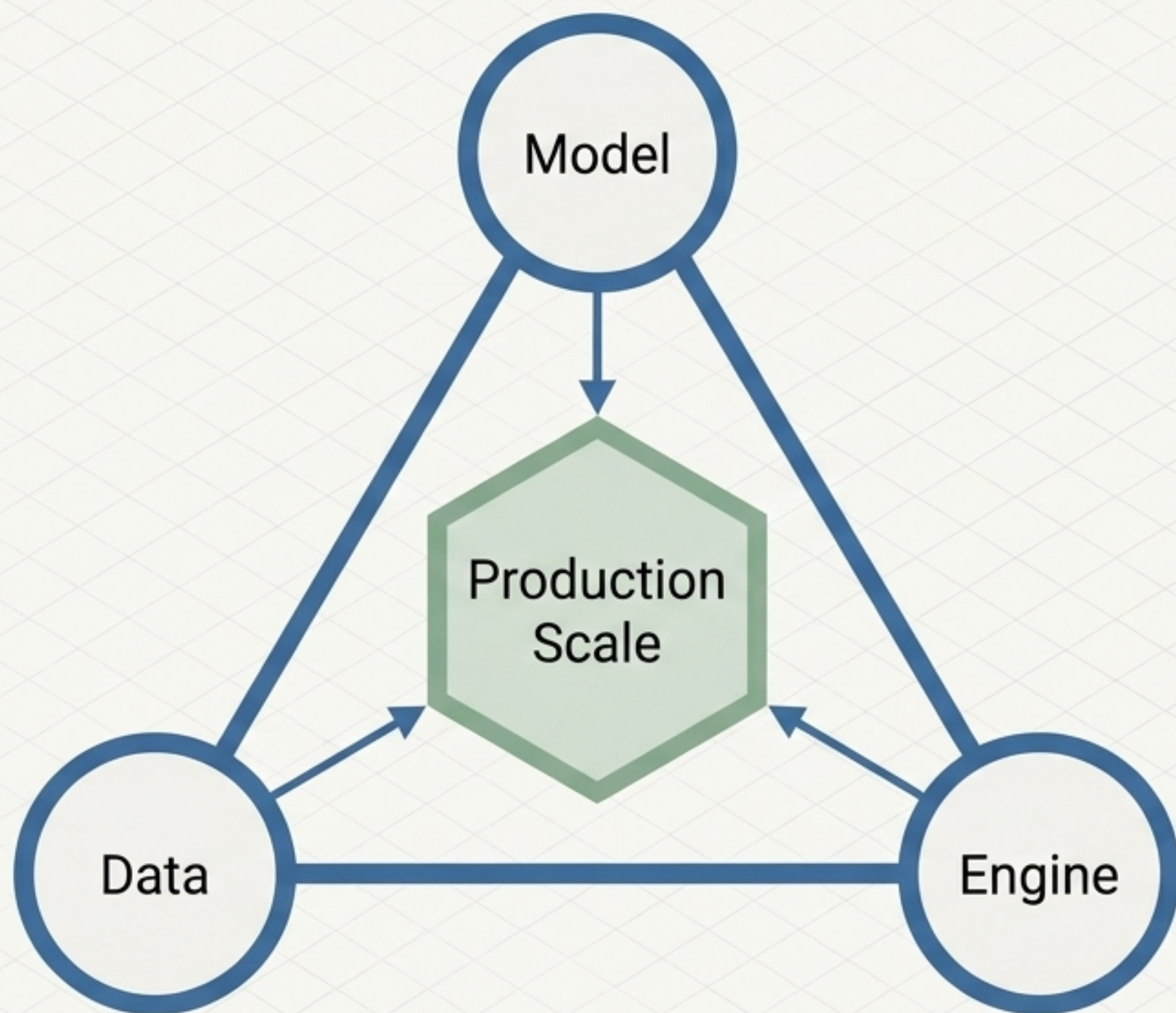
-7.15%

(แนะนำงานได้ตรงใจ ลดการปิดทิ้ง)

Insight: โมเดลที่ถูกบีบอัดให้เล็กที่สุด กลับมอบประสบการณ์การหางานที่ดีที่สุดในระดับโปรดักชัน

พิมพ์เขียวสำหรับ Production SLM

System Status: Blueprint Complete



เพื่อนำ Generative AI ขึ้นสู่ระดับโปรดัคชันที่รองรับโหลดมหาศาล คุณไม่สามารถพึ่งพาวิธีใดวิธีหนึ่งได้

Co-Design is Mandatory: ต้องออกแบบ Model (ให้เล็ก), Data (ให้กระชับ), และ Engine (ให้เฉพาะทาง) ไปพร้อมๆ กัน

อย่าปล่อยให้ทรัพยากรสูญเปล่าไปกับการประมวลผลที่ระบบไม่ได้นำไปใช้